

Sistemas embebidos

Modos de direccionamiento

μ C ATmega328P

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2025-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

¿Qué son los Modos de Direccionamiento?

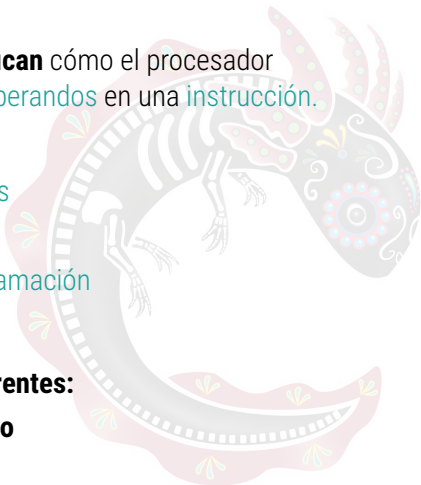
Los modos de direccionamiento **especifican** cómo el procesador **determina la dirección** efectiva de los **operandos** en una **instrucción**.

¿Por qué son importantes?

- Definen **cómo se accede** a los **datos**
- Afectan la **eficiencia** del **programa**
- Determinan la **flexibilidad** de **programación**
- Influyen en el **tamaño** del **código**

El ATmega328P soporta 13 modos diferentes:

- **Registro directo, Inmediato, Directo**
- **Indirecto (con variaciones)**
- **Relativo, Implícito**



Espacios de Memoria del ATmega328P

- **Flash (32KB):** Programa + constantes
- **SRAM (2KB):** Variables + stack
- **EEPROM (1KB):** Datos no volátiles
- **Registros (32):** Procesamiento rápido
- **E/S (224):** Periféricos



Nota Importante

Los diferentes **modos de direccionamiento** acceden a **diferentes espacios** de memoria

Registro Directo - Definición

Los operandos son **registros del procesador** (R0-R31). La **dirección efectiva** es el **número del registro** especificado en la instrucción.

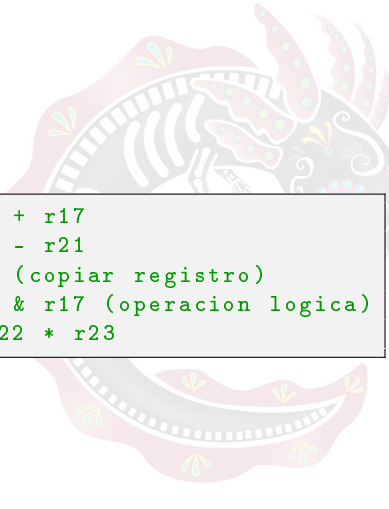
Características:

- **Más rápido:** Acceso directo a **registros internos**
- **1 ciclo:** Tiempo de ejecución **mínimo**
- **Compacto:** Solo requiere **5 bits** para direccionar **32 registros**
- **Limitado:** Solo **32 ubicaciones** disponibles



Registro Directo - Definición

Ejemplos:



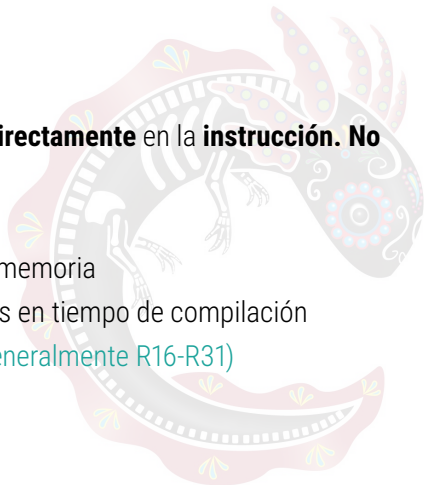
```
1 ADD r16, r17      ; r16 = r16 + r17
2 SUB r20, r21      ; r20 = r20 - r21
3 MOV r18, r19      ; r18 = r19 (copiar registro)
4 AND r16, r17      ; r16 = r16 & r17 (operacion logica)
5 MUL r22, r23      ; r1:r0 = r22 * r23
```

Direccionamiento Inmediato

El operando es una **constante** incluida **directamente** en la **instrucción**. **No hay acceso** a **memoria adicional**.

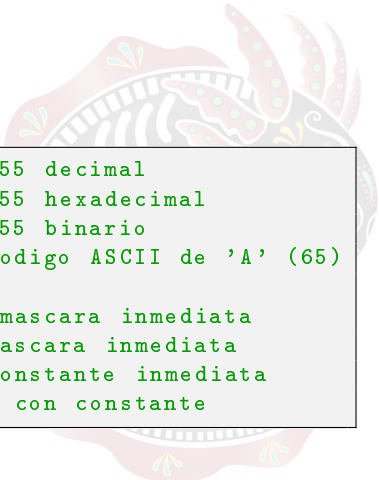
Características:

- **Rápido:** Sin accesos adicionales a memoria
- **Constantes:** Valores fijos conocidos en tiempo de compilación
- **Limitado:** Solo ciertos registros (generalmente R16-R31)
- **Rango:** Típicamente 0-255 (8 bits)



Direccionamiento Inmediato

Ejemplos:



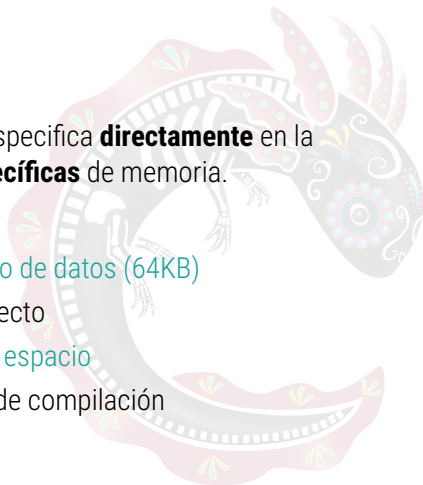
```
1 LDI r16, 255           ; Cargar 255 decimal
2 LDI r17, 0xFF          ; Cargar 255 hexadecimal
3 LDI r18, 0b11111111    ; Cargar 255 binario
4 LDI r19, 'A'           ; Cargar codigo ASCII de 'A' (65)
5
6 ANDI r16, 0xF0          ; AND con mascara inmediata
7 ORI r17, 0x0F           ; OR con mascara inmediata
8 SUBI r18, 10            ; Restar constante inmediata
9 CPI r19, 100            ; Comparar con constante
```

Direccionamiento Directo

La **dirección efectiva** del operando se especifica **directamente** en la **instrucción**. Acceso a **ubicaciones específicas** de memoria.

Características:

- **Acceso completo:** A todo el **espacio de datos (64KB)**
- **2 ciclos:** **Más lento** que registro directo
- **2 palabras:** Instrucción **ocupa más espacio**
- **Absoluto:** **Dirección fija** en tiempo de compilación



Direccionamiento Directo

Ejemplo:

```

1 LDS r16, 0x0100      ; Load Direct from SRAM
2 STS 0x0100, r16      ; Store Direct to SRAM
3
4 ; Acceso a registros de E/S extendida
5 LDS r17, 0x0080      ; Leer registro extendido
6 STS 0x0080, r17      ; Escribir registro extendido
7
8 ; Variables en SRAM
9 .dseg
10 variable: .byte 1     ; Definir variable
11 presion: .byte 2      ; Variable de 2 bytes
12
13 .cseg
14 LDS r16, variable     ; Leer variable
15 STS variable, r16     ; Escribir variable
16 LDS r17, presion      ; Leer parte baja de presion
17 LDS r18, presion+1    ; Leer parte alta de presion

```

Direccionamiento Indirecto

La **dirección efectiva** está **almacenada** en un **registro apuntador**. El **contenido** del apuntador se usa como **dirección** para acceder al operando.

Apuntadores disponibles en ATmega328P:

- **X (R27:R26):** apuntador de 16 bits para SRAM
- **Y (R29:R28):** apuntador con desplazamiento (+q)
- **Z (R31:R30):** apuntador para Flash, SRAM y EEPROM

Ventajas:

- **Flexibilidad:** Dirección calculada en tiempo de ejecución
- **Arrays:** Acceso secuencial eficiente
- **Estructuras:** Navegación por datos complejos
- **Algoritmos:** Apuntadores son esenciales para muchos algoritmos

Indirecto - Apuntador X (R27:R26)

Ejemplo: Apuntador básico (sin desplazamiento)

```
1 ; Configurar apuntador X
2 LDI XL, low(buffer) ; Parte baja de la direccion
3 LDI XH, high(buffer); Parte alta de la direccion
4
5 ; Operaciones basicas
6 LD r16, X           ; Cargar desde direccion apuntada por X
7 ST X, r16           ; Almacenar en direccion apuntada por X
8
9 ; Post-incremento (muy util para arrays)
10 LD r16, X+          ; Cargar y luego X = X + 1
11 ST X+, r16          ; Almacenar y luego X = X + 1
12
13 ; Pre-decremento
14 LD r16, -X           ; X = X - 1, luego cargar
15 ST -X, r16          ; X = X - 1, luego almacenar
```

Indirecto - Apuntador Y (R29:R28)

Ejemplo: Apuntador con capacidad de desplazamiento:

```
1 ; Configurar apuntador Y
2 LDI YL, low(estructura)
3 LDI YH, high(estructura)
4
5 ; Operaciones basicas (como X)
6 LD r16, Y ; Cargar desde Y
7 ST Y, r16 ; Almacenar en Y
8 LD r16, Y+ ; Cargar y post-incrementar
9 ST -Y, r16 ; Pre-decrementar y almacenar
10
11 ; Caracteristica especial: desplazamiento (0-63)
12 LD r16, Y+5 ; Cargar desde Y + 5
13 ST Y+10, r16 ; Almacenar en Y + 10
```

Indirecto - Apuntador Z (R31:R30)

Ejemplo: Apuntador con acceso a SRAM (como X e Y):

```
1 LD r16, Z           ; Cargar desde SRAM
2 ST Z, r16           ; Almacenar en SRAM
3 LD r16, Z+          ; Con post-incremento
4 ST -Z, r16          ; Con pre-decremento
5 LD r16, Z+q         ; Con desplazamiento (como Y)
```

Indirecto - Apuntador Z (R31:R30)

Ejemplo: Apuntador con acceso a memoria Flash (solo lectura):

```

1  LPM r16, Z                ; Load Program Memory
2  LPM r16, Z+               ; Con post-incremento
3
4  ; Ejemplo - Leer tabla de constantes
5  .cseg
6  tabla_senos: .db 0, 25, 50, 71, 87, 96, 100, 96, 87, 71,
   50, 25
7
8  main:
9      ; Direccion en palabras (*2)
10     LDI ZL, low(tabla_senos*2)
11     LDI ZH, high(tabla_senos*2)
12
13     LPM r16, Z+            ; Leer primer valor (0)
14     LPM r17, Z+            ; Leer segundo valor (25)
15     LPM r18, Z+            ; Leer tercer valor (50)

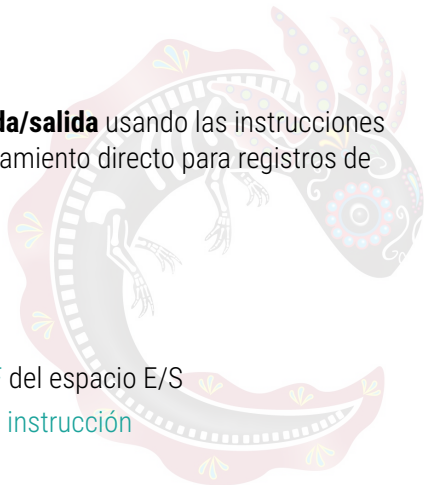
```

Direccionamiento E/S Directo

Acceso directo a los **registros** de **entrada/salida** usando las instrucciones **IN** y **OUT**. **Más eficiente** que el direccionamiento directo para registros de E/S.

Características:

- **Rápido:** 1 ciclo de reloj
- **Compacto:** 1 palabra de programa
- **Limitado:** Solo registros 0x00-0x3F del espacio E/S
- **Directo:** Dirección del registro en la instrucción



Ejemplos:

```
1 ; Configuracion de puertos
2 OUT DDRB, r16          ; Configurar direccion Puerto B
3 OUT PORTB, r17         ; Escribir datos a Puerto B
4 IN r18, PINB           ; Leer estado Puerto B
5
6 ; Otros registros importantes
7 IN r19, SREG           ; Leer registro de estado
8 OUT SREG, r19          ; Escribir registro de estado (
    cuidado!)
9
10 IN r20, SPL            ; Leer Stack Pointer Low
11 OUT SPL, r20           ; Escribir Stack Pointer Low
```


E/S Directo vs Directo - Comparación

Para registros 0x00-0x3F - Usar IN/OUT:

```

1 ; Eficiente - 1 ciclo, 1 palabra
2 IN r16, PORTB           ; Leer PORTB
3 OUT PORTB, r16          ; Escribir PORTB
4
5 ; Ineficiente - 2 ciclos, 2 palabras
6 LDS r16, 0x25            ; PORTB esta en 0x25 del espacio de
   datos
7 STS 0x25, r16            ; (0x20 + direccion E/S 0x05)

```

Para registros 0x60-0xFF - Usar LDS/STS:

```

1 ; Registros extendidos solo accesibles con LDS/STS
2 LDS r16, UDRO            ; UART Data Register
3 STS UDRO, r16            ; No existe OUT UDRO, r16
4
5 LDS r16, TCNT1L          ; Timer1 Counter Low
6 LDS r17, TCNT1H          ; Timer1 Counter High

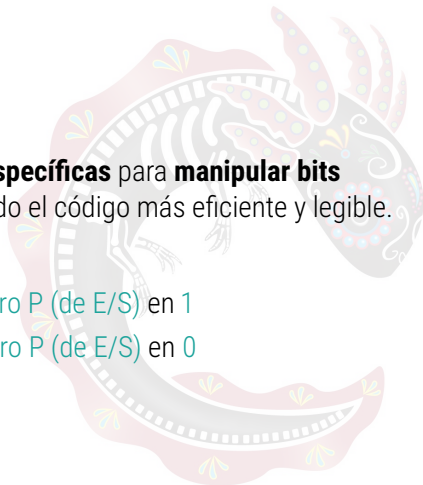
```

Manipulación de Bits en E/S

El ATmega328P incluye **instrucciones específicas** para **manipular bits** individuales en **registros de E/S**, haciendo el código más eficiente y legible.

Instrucciones disponibles:

- **SBI P,b:** establece el bit b del registro P (de E/S) en 1
- **CBI P,b:** establece el bit b del registro P (de E/S) en 0



Manipulación de Bits en E/S

Ejemplo: Control de LEDs

```
1 ; Configurar pin como salida
2 SBI DDRB, 0 ; PBO como salida
3
4 ; Encender LED
5 SBI PORTB, 0 ; Encender LED en PBO
6
7 ; Apagar LED
8 CBI PORTB, 0 ; Apagar LED en PBO
9
10 ; Toggle LED (requiere leer-modificar-escribir)
11 IN r16, PORTB ; Leer estado actual
12 LDI r17, (1<<0) ; Mascara para bit 0
13 EOR r16, r17 ; XOR para alternar bit
14 OUT PORTB, r16 ; Escribir nuevo estado
```

Direccionamiento Relativo

La **dirección efectiva** se **calcula sumando** un **desplazamiento (offset)** al **Program Counter (PC)** actual. Usado principalmente en **instrucciones de salto**.

Características:

- **Código relocatable:** Independiente de la posición absoluta
- **Compacto:** Menos bits necesarios para el desplazamiento
- **Rango limitado:** Depende de la instrucción específica
- **Calculado automáticamente:** El ensamblador calcula el offset

Direccionamiento Relativo

Ejemplos:

```

1 ; Saltos relativos
2     RJMP main          ; Rango: +-2048 palabras
3     RCALL funcion      ; Rango: +-2048 palabras
4
5 ; Saltos condicionales relativos
6     BREQ etiqueta     ; Rango: +-64 palabras
7     BRNE bucle         ; Rango: +-64 palabras
8     BRCS overflow     ; Rango: +-64 palabras
9
10 ; El ensamblador calcula automaticamente el offset
11 main:                  ; PC = 0x100 (ejemplo)
12     LDI r16, 10
13     RJMP bucle         ; Offset calculado automaticamente
14     NOP
15 bucle:                 ; PC = 0x103
16     DEC r16
17     BRNE bucle         ; Offset = -2 (volver 2 inst.)
  
```

Relativo - Rangos y Limitaciones

Rangos de diferentes instrucciones:

Instrucción	Rango	Uso
RJMP	± 2048 palabras	Salto grandes
RCALL	± 2048 palabras	Llamadas a subrutinas
BRxx	± 64 palabras	Salto condicionales

Ventajas del código relocatable:

```

1 ; Este codigo funciona en cualquier posicion
2 funcion_portable:
3     LDI r16, 10
4 bucle_local: ; Siempre funciona sin importar donde este
5     DEC r16
6     BRNE bucle_local
7     RET

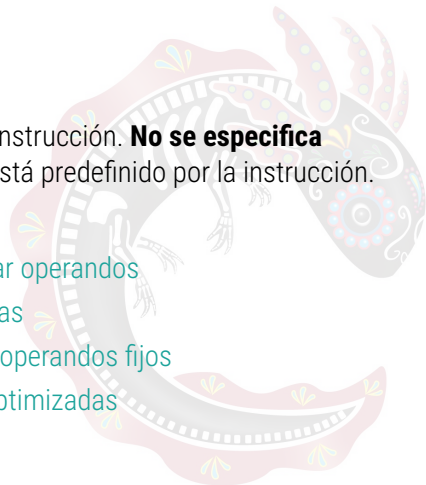
```

Direccionamiento Implícito

El **operando** está **implícito** en la propia instrucción. **No se especifica** explícitamente la **dirección** o **registro**, está predefinido por la instrucción.

Características:

- **Automático:** No requiere especificar operandos
- **Compacto:** Instrucciones muy cortas
- **Específico:** Cada instrucción tiene operandos fijos
- **Eficiente:** Operaciones comunes optimizadas



Direccionamiento Implícito

Ejemplos comunes:

```

1 ; Operaciones con resultado implícito en R1:R0
2 MUL r16, r17 ; Resultado siempre en R1:R0
3
4 ; Operaciones del stack (SP implícito)
5 PUSH r16 ; Usar SP automáticamente
6 POP r17 ; Usar SP automáticamente
7
8 ; Operaciones con registros fijos
9 LPM r16, Z ; Usar Z como apuntador implícito
10
11 ; Control de interrupciones (SREG implícito)
12 SEI ; Set Enable Interrupts (I=1 en SREG)
13 CLI ; Clear Interrupts (I=0 en SREG)
14
15 ; Operaciones especiales
16 NOP ; No Operation (PC = PC + 1)
17 WDR ; Watchdog Reset

```


Comparación de Rendimiento



Modo	Ciclos	Palabras	Rango	Mejor Uso
Registro Directo	1	1	32 reg.	Operaciones frecuentes
Inmediato	1	1	0-255	Constantes conocidas
E/S Directo	1	1	64 reg.	Periféricos estándar
Directo	2	2	64KB	Variables globales
Indirecto	1-2	1	64KB	Arrays, estructuras
Relativo	1-2	1	Variable	Saltos y llamadas
Implícito	1-4	1	N/A	Operaciones específicas

Comparación de Rendimiento

Recomendaciones de Optimización

- **Registro Directo:** Para **operaciones críticas** en tiempo
- **E/S Directo:** Preferir **IN/OUT** sobre **LDS/STS** para **E/S estándar**
- **Indirecto:** Con **post-incremento** para arrays secuenciales
- **Inmediato:** Para **constantes pequeñas** (<256)

Técnicas de Optimización

1. Elección del modo apropiado:

```

1 ; Ineficiente
2 LDS r16, 0x25           ; 2 ciclos para acceder a PORTB
3 STS 0x25, r16           ; 2 ciclos
4
5 ; Eficiente
6 IN r16, PORTB           ; 1 ciclo
7 OUT PORTB, r16          ; 1 ciclo

```

2. Aprovechamiento de registros especiales:

```

1 ; Usar apuntador Y para estructuras con offsets
2 LD r16, Y+0             ; Campo 0
3 LD r17, Y+1             ; Campo 1
4 LD r18, Y+5             ; Campo 5 (saltar campos)

```

Técnicas de Optimización

3. Reutilización de apuntadores:

```
1 ; Recalcular apuntador cada vez
2 LDI XL, low(buffer)
3 LDI XH, high(buffer)
4 LD r16, X
5 ; ... otras operaciones ...
6 LDI XL, low(buffer+1) ; Recalcular
7 LDI XH, high(buffer+1)
8 LD r17, X
9
10 ; Usar post-incremento
11 LDI XL, low(buffer)
12 LDI XH, high(buffer)
13 LD r16, X+ ; Automatico incremento
14 LD r17, X+ ; Siguiente posicion
```

Resumen de Modos de Direcccionamiento

Modo	Descripción	Ejemplo	Ciclos
Registro Directo	Operandos en registros	ADD r16, r17	1
Inmediato	Constante en instrucción	LDI r16, 100	1
Directo	Dirección absoluta	LDS r16, 0x100	2
E/S Directo	Registro E/S directo	IN r16, PORTB	1
Indirecto	A través de apuntador	LD r16, X	1-2
Indirecto + Despl.	apuntador + offset	LD r16, Y+5	2
Indirecto + Post-Inc	apuntador, luego inc	LD r16, X+	2
Indirecto + Pre-Dec	Dec, luego apuntador	LD r16, -X	2
Relativo	PC + desplazamiento	RJMP etiqueta	2
Implícito	Operando predefinido	PUSH r16	2

Guía de Selección de Modos

Registro Directo:

- Operaciones frecuentes y críticas en tiempo
- Variables temporales y contadores

Inmediato:

- Constantes conocidas en compilación
- Configuración inicial de registros
- Máscaras y valores de configuración

Directo:

- Variables globales en SRAM
- Registros extendidos de E/S
- Configuración poco frecuente



Guía de Selección de Modos

Indirecto:

- Arrays y buffers
- Estructuras de datos complejas
- Algoritmos con **apuntadores**
- Tablas de **lookup** dinámicas



Optimización de Rendimiento

- 1 **Preferir registro directo** para operaciones frecuentes
- 2 **Usar IN/OUT** en lugar de LDS/STS para E/S estándar
- 3 **Aprovechar post-incremento** en procesamiento secuencial
- 4 **Reutilizar apuntadores** en lugar de recalcular direcciones

Mejores Prácticas

Legibilidad del Código

- 1 **Usar .def** para asignar nombres a registros
- 2 **Comentar** el propósito de cada apuntador
- 3 **Agrupar** variables relacionadas en **SRAM**
- 4 **Usar constantes simbólicas** en lugar de números mágicos

Robustez

- 1 **Verificar límites** en acceso indirecto
- 2 **Inicializar apuntadores** antes de usar
- 3 **Manejar casos especiales** (buffers llenos, etc.)
- 4 **Usar stack** para preservar registros en interrupciones

